

Berikut adalah **Quiz 10 Soal** yang mencakup materi **Minggu 1: Pendahuluan dan Kerangka Kerja Probabilitas**, yang didasarkan pada konsep-konsep seperti Ruang Sampel, Kejadian (*Event*), Operasi Himpunan (Union, Intersection, Komplemen), dan perhitungan probabilitas dasar menggunakan data tabel,.

Setiap soal disertakan dengan solusi menggunakan bahasa pemrograman **Python**.

Quiz Minggu 1: Konsep Dasar Probabilitas

Soal 1: Ruang Sampel Eksperimen Biner

Sebuah sistem komunikasi mengirimkan sinyal biner (0 atau 1) sebanyak 3 kali berturut-turut. Tuliskan kode Python untuk men-generate seluruh ruang sampel (semua kemungkinan hasil) dari eksperimen ini.

Solusi Python:

```
import itertools

# Mendefinisikan kemungkinan hasil untuk satu sinyal
sinyal =

# Menghasilkan Cartesian product untuk 3 pengiriman
ruang_sampel = list(itertools.product(sinyal, repeat=3))

print(f"Ruang Sampel (Total {len(ruang_sampel)} elemen):")
for hasil in ruang_sampel:
    print(hasil)
```

Soal 2: Probabilitas Gabungan (Union)

Diketahui probabilitas kejadian A adalah $P(A) = 0.6$, probabilitas kejadian B adalah $P(B) = 0.5$, dan probabilitas irisan keduanya $P(A \cap B) = 0.3$. Hitunglah $P(A \cup B)$.

Solusi Python:

```
P_A = 0.6
P_B = 0.5
P_A_irisan_B = 0.3

# Menggunakan rumus  $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ 
P_A_union_B = P_A + P_B - P_A_irisan_B

print(f"Probabilitas  $P(A \cup B)$  adalah: {P_A_union_B:.2f}")
```

Soal 3: Probabilitas Komplemen

Dalam sebuah pengujian sirkuit, probabilitas sirkuit lolos uji adalah 0.85. Tentukan probabilitas sirkuit tersebut **gagal** (tidak lolos uji).

Solusi Python:

```
P_Lolos = 0.85

# P(Ac) = 1 - P(A)
P_Gagal = 1 - P_Lolos

print(f"Probabilitas Gagal (Komplemen): {P_Gagal:.2f}")
```

Soal 4: Kejadian Saling Eksklusif (Mutually Exclusive)

Terdapat tiga kejadian A, B, dan C. Diketahui $P(A) = 0.4$, $P(B) = 0.35$, dan $P(C) = 0.3$. Tentukan apakah ketiga kejadian tersebut bisa bersifat *mutually exclusive* (saling asing) satu sama lain dalam satu ruang sampel yang sama? Buat program untuk mengeceknya.

Solusi Python:

```
P_A = 0.4
P_B = 0.35
P_C = 0.3

def cek_mutually_exclusive(probs):
    total_prob = sum(probs)
    # Jika total probabilitas > 1, maka tidak mungkin mutually exclusive dalam
    satu ruang sampel
    if total_prob <= 1:
        return True, total_prob
    else:
        return False, total_prob

is_possible, total = cek_mutually_exclusive([P_A, P_B, P_C])
print(f"Total Probabilitas: {total}")
print(f"Apakah mungkin Mutually Exclusive? {'Ya' if is_possible else 'Tidak'}")
```

Soal 5: Analisis Data Tabel (Joint Probability)

Berdasarkan data sampel 100 piringan plastik: - Tahan Gores Tinggi & Tahan Guncang Tinggi = 70 - Tahan Gores Tinggi & Tahan Guncang Rendah = 9 - Tahan Gores Rendah & Tahan Guncang Tinggi = 16 - Tahan Gores Rendah & Tahan Guncang Rendah = 5

Hitung probabilitas sebuah piringan yang dipilih secara acak memiliki **Ketahanan Gores Tinggi** **DAN Ketahanan Guncang Tinggi**.

Solusi Python:

```
# Data
jumlah_sampel = 100
gores_tinggi_guncang_tinggi = 70

# Probabilitas Joint
probabilitas = goes_tinggi_guncang_tinggi / jumlah_sampel

print(f"P(Gores Tinggi n Guncang Tinggi) = {probabilitas}")
```

Soal 6: Marginal Probability dari Tabel

Menggunakan data yang sama dengan Soal 5, hitunglah probabilitas piringan memiliki **Ketahanan Guncang Rendah** (tanpa mempedulikan ketahanan goresnya).

Solusi Python:

```
# Data kejadian yang relevan
gores_tinggi_guncang_rendah = 9
gores_rendah_guncang_rendah = 5
total_sampel = 100

# Menjumlahkan semua kejadian guncang rendah (Marginal Probability)
total_guncang_rendah = goes_tinggi_guncang_rendah +
gores_rendah_guncang_rendah
prob_guncang_rendah = total_guncang_rendah / total_sampel

print(f"P(Guncang Rendah) = {prob_guncang_rendah}")
```

Soal 7: Hukum De Morgan (Probabilitas)

Diketahui $P(A \cup B) = 0.8$. Hitunglah probabilitas kejadian di mana **tidak A DAN tidak B** terjadi ($P(A^c \cap B^c)$).

Solusi Python:

```
P_A_union_B = 0.8

# Berdasarkan hukum De Morgan:  $P(A^c \cap B^c) = P((A \cup B)^c) = 1 - P(A \cup B)$ 
P_Ac_irisan_Bc = 1 - P_A_union_B
```

```
print(f"P(Tidak A dan Tidak B) = {P_Ac_irisan_Bc:.2f}")
```

Soal 8: Conditional Probability Sederhana

Dalam sebuah survei terhadap 200 orang, 150 orang menyukai Kopi (A), 100 orang menyukai Teh (B), dan 80 orang menyukai keduanya. Hitung probabilitas seseorang menyukai Teh **jika diketahui** dia menyukai Kopi ($P(B | A)$).

Solusi Python:

```
n_A = 150 # Suka Kopi
n_B = 100 # Suka Teh
n_A_irisan_B = 80 # Suka Keduanya

#  $P(B|A) = n(A \cap B) / n(A)$ 
prob_B_given_A = n_A_irisan_B / n_A

print(f"P(Suka Teh | Suka Kopi) = {prob_B_given_A:.2f}")
```

Soal 9: Ukuran Ruang Sampel (Papan Sirkuit)

Sebuah papan sirkuit diuji. Hasilnya bisa **Lolos** atau **Gagal**. Jika Gagal, sirkuit diperiksa untuk menentukan salah satu dari **5 jenis cacat** yang mungkin. Hitung berapa total titik sampel (*outcome*) yang mungkin terjadi dari eksperimen ini.

Solusi Python:

```
# Kejadian 1: Lolos
kejadian_lolos = 1

# Kejadian 2: Gagal (dengan 5 sub-kejadian jenis cacat)
kejadian_gagal = 5

total_titik_sampel = kejadian_lolos + kejadian_gagal

print(f"Total titik sampel yang mungkin: {total_titik_sampel}")
# Hasilnya adalah {Lolos, Gagal_1, Gagal_2, Gagal_3, Gagal_4, Gagal_5}
```

Soal 10: Frekuensi Relatif

Diberikan data waktu respon server (dalam ms): ``. Hitung probabilitas empiris (frekuensi relatif) bahwa waktu respon server **lebih dari 28 ms**.

Solusi Python:

```
data_respon =

# Menghitung jumlah kejadian > 28
kejadian_lebih_28 = [x for x in data_respon if x > 28]
jumlah_kejadian = len(kejadian_lebih_28)
total_data = len(data_respon)

probabilitas = jumlah_kejadian / total_data

print(f"Probabilitas waktu respon > 28 ms: {probabilitas}")
```

Tentu, ini adalah versi simulasi Monte Carlo untuk ke-10 soal tersebut. Pendekatan Monte Carlo di sini menggunakan pembangkitan bilangan acak (*random sampling*) dalam jumlah besar untuk mendekati probabilitas teoritis.

Untuk setiap soal, kita akan melakukan eksperimen berulang kali (misalnya 100.000 atau 1.000.000 kali) dan menghitung proporsi kejadian yang memenuhi syarat.

Simulasi Monte Carlo: Quiz Minggu 1

Soal 1: Simulasi Ruang Sampel Eksperimen Biner

Meskipun Monte Carlo biasanya untuk menghitung nilai, kita bisa menggunakannya untuk *mencari* ruang sampel dengan men-generate banyak sampel acak dan menyimpan hasil yang unik.

```
import random

def monte_carlo_sample_space(num_trials=1000):
    unique_outcomes = set()

    for _ in range(num_trials):
        # Simulasi 3 sinyal (0 atau 1)
        outcome = tuple(random.choices(0, 1, k=3))
        unique_outcomes.add(outcome)

    # Mengurutkan hasil agar rapi
    sorted_outcomes = sorted(list(unique_outcomes))
    print(f"Ruang Sampel Ditemukan (Total {len(sorted_outcomes)}):")
    for out in sorted_outcomes:
        print(out)

monte_carlo_sample_space()
```

Soal 2: Simulasi Probabilitas Gabungan (Union)

Kita akan mensimulasikan kejadian A dan B berdasarkan probabilitas yang diketahui. Karena $P(A \cap B)$ ditentukan, kita harus membangkitkan data yang berkorelasi sesuai struktur tersebut. *Strategi:* Kita bagi rentang $[0, 1]$ menjadi bagian-bagian eksklusif: hanya A, hanya B, keduanya, dan tidak keduanya. * $P(\text{hanya A}) = P(A) - P(A \cap B) = 0.6 - 0.3 = 0.3$ * $P(\text{hanya B}) = P(B) - P(A \cap B) = 0.5 - 0.3 = 0.2$ * $P(A \cap B) = 0.3$ * $P(\text{Lainnya}) = 1 - (0.3 + 0.2 + 0.3) = 0.2$

```
import random

def monte_carlo_union(trials=100000):
    count_union = 0

    for _ in range(trials):
        r = random.random() # Bilangan acak 0 s.d 1

        # Logika pembagian area probabilitas
        if r < 0.3:          # Hanya A
            event_A = True
            event_B = False
        elif r < 0.5:        # Hanya B (0.3 + 0.2)
            event_A = False
            event_B = True
        elif r < 0.8:        # A dan B (0.5 + 0.3)
            event_A = True
            event_B = True
        else:                # Tidak keduanya
            event_A = False
            event_B = False

        # Cek Union (A atau B terjadi)
        if event_A or event_B:
            count_union += 1

    print(f"Simulasi P(A U B): {count_union / trials:.4f}")
    print("Teoritis P(A U B): 0.80")

monte_carlo_union()
```

Soal 3: Simulasi Probabilitas Komplemen

Simulasi sederhana dengan *Bernoulli Trial*.

```

import random

def monte_carlo_complement(trials=100000):
    p_lolos = 0.85
    count_gagal = 0

    for _ in range(trials):
        # Generate hasil tes (Lolos jika random < 0.85)
        if random.random() >= p_lolos:
            count_gagal += 1

    print(f"Simulasi P(Gagal): {count_gagal / trials:.4f}")
    print(f"Teoritis P(Gagal): {1 - p_lolos:.2f}")

monte_carlo_complement()

```

Soal 4: Simulasi Mutually Exclusive

Kita akan mencoba menempatkan kejadian A, B, dan C pada satu garis bilangan \$\$ tanpa tumpang tindih. Jika total panjang > 1, simulasi akan menunjukkan “tabrakan” atau ketidakmungkinan.

```

import random

def monte_carlo_mutually_exclusive(probs, trials=100000):
    total_req = sum(probs)
    if total_req > 1:
        print("Simulasi: Tidak mungkin Mutually Exclusive (Total Probabilitas > 1)")
        return

    # Jika mungkin, kita simulasi kejadiannya
    counts = {i: 0 for i in range(len(probs))}

    # Membuat batas kumulatif
    boundaries = []
    current = 0
    for p in probs:
        current += p
        boundaries.append(current)

    for _ in range(trials):
        r = random.random()
        for i, bound in enumerate(boundaries):

```

```

        if r < bound:
            counts[i] += 1
            break # Karena mutually exclusive, hanya satu yang bisa terjadi

print("Simulasi Frekuensi Relatif:")
for i, count in counts.items():
    print(f"Event {i}: {count/trials:.4f} (Target: {probs[i]})")

# Test Case
monte_carlo_mutually_exclusive([0.4, 0.35, 0.3])

```

Soal 5: Simulasi Data Tabel (Joint Probability)

Kita akan membuat “populasi buatan” berdasarkan data tabel, lalu mengambil sampel secara acak (*sampling*).

```

import random

def monte_carlo_joint_prob(trials=100000):
    # Membuat populasi berdasarkan data tabel
    # Format: (Gores, Guncang) -> (Tinggi=1/Rendah=0, Tinggi=1/Rendah=0)
    populasi = []
    populasi.extend([(1, 1)] * 70) # Gores T, Guncang T
    populasi.extend([(1, 0)] * 9)  # Gores T, Guncang R
    populasi.extend([(0, 1)] * 16) # Gores R, Guncang T
    populasi.extend([(0, 0)] * 5)  # Gores R, Guncang R

    count_event = 0

    for _ in range(trials):
        # Ambil satu sampel secara acak
        sampel = random.choice(populasi)

        # Cek kondisi: Gores Tinggi (1) DAN Guncang Tinggi (1)
        if sampel == (1, 1):
            count_event += 1

    print(f"Simulasi Joint Probability: {count_event / trials:.4f}")
    print("Teoritis: 0.70")

monte_carlo_joint_prob()

```

Soal 6: Simulasi Marginal Probability

Menggunakan populasi yang sama dengan Soal 5, tapi kondisi pengecekannya berbeda.

```
import random

def monte_carlo_marginal(trials=100000):
    # Populasi sama seperti di atas
    populasi = []
    populasi.extend([(1, 1)] * 70)
    populasi.extend([(1, 0)] * 9)
    populasi.extend([(0, 1)] * 16)
    populasi.extend([(0, 0)] * 5)

    count_guncang_rendah = 0

    for _ in range(trials):
        sampel = random.choice(populasi)
        # Indeks 1 adalah ketahanan guncang (0 = Rendah)
        if sampel == 0:
            count_guncang_rendah += 1

    print(f"Simulasi P(Guncang Rendah): {count_guncang_rendah / trials:.4f}")
    print("Teoritis: 0.14")

monte_carlo_marginal()
```

Soal 7: Simulasi Hukum De Morgan

Simulasi ini mirip dengan Soal 2, kita mendefinisikan area probabilitas. Diketahui $P(A \cup B) = 0.8$. Artinya probabilitas “di luar” A atau B adalah $1 - 0.8 = 0.2$.

```
import random

def monte_carlo_demorgan(trials=100000):
    # Kita asumsikan P(A U B) menempati rentang 0.0 s.d 0.8
    # Maka P(A' n B') adalah area > 0.8
    limit_union = 0.8
    count_notA_and_notB = 0

    for _ in range(trials):
        r = random.random()

        # Logika: Jika r <= 0.8, maka dia masuk area Union (Entah A, Entah B,
```

```

atau Keduanya)
    is_in_union = r <= limit_union

    # De Morgan: Tidak A DAN Tidak B adalah komplemen dari Union
    if not is_in_union:
        count_notA_and_notB += 1

    print(f"Simulasi P(A' n B'): {count_notA_and_notB / trials:.4f}")
    print("Teoritis: 0.20")

monte_carlo_demorgan()

```

Soal 8: Simulasi Conditional Probability

Untuk menghitung $P(B | A)$, kita menggunakan teknik *rejection sampling*. Kita hanya menghitung percobaan di mana kejadian syarat (A) terjadi.

```

import random

def monte_carlo_conditional(trials=100000):
    # Representasi Populasi 200 orang
    # A = Kopi, B = Teh
    populasi = []

    # 80 Suka Keduanya (A=1, B=1)
    populasi.extend([(1, 1)] * 80)

    # 70 Suka Kopi Saja (A=1, B=0) -> Total Kopi 150 (150-80=70)
    populasi.extend([(1, 0)] * 70)

    # 20 Suka Teh Saja (A=0, B=1) -> Total Teh 100 (100-80=20)
    populasi.extend([(0, 1)] * 20)

    # 30 Tidak Suka Keduanya (Sisa populasi: 200 - 80 - 70 - 20)
    populasi.extend([(0, 0)] * 30)

    count_A_occurred = 0
    count_B_given_A = 0

    for _ in range(trials):
        sampel = random.choice(populasi)

        # Syarat: Diketahui Suka Kopi (A=1)

```

```

        if sampel == 1:
            count_A_occurred += 1
            # Cek apakah juga suka Teh (B=1)
            if sampel == 1:
                count_B_given_A += 1

    prob = count_B_given_A / count_A_occurred
    print(f"Simulasi P(Teh | Kopi): {prob:.4f}")
    print(f"Teoritis: {80/150:.4f}")

monte_carlo_conditional()

```

Soal 9: Simulasi Pembangkitan Titik Sampel

Mirip dengan soal 1, kita men-generate kejadian secara acak untuk melihat variasi hasil yang mungkin terjadi.

```

import random

def monte_carlo_sample_points(trials=1000):
    outcomes = set()

    for _ in range(trials):
        # Tahap 1: Uji Lolos/Gagal
        # Asumsi probabilitas 50:50 untuk tujuan eksplorasi ruang sampel
        status = random.choice(['Lolos', 'Gagal'])

        if status == 'Lolos':
            outcomes.add('Lolos')
        else:
            # Tahap 2: Jika Gagal, cek jenis cacat (1-5)
            cacat = random.randint(1, 5)
            outcomes.add(f'Gagal_Tipe_{cacat}')

    print(f"Titik Sampel ditemukan ({len(outcomes)}):")
    print(sorted(list(outcomes)))

monte_carlo_sample_points()

```

Soal 10: Simulasi Bootstrap (Resampling)

Untuk data empiris yang kecil, teknik Monte Carlo yang relevan adalah *Bootstrap Resampling*. Kita mengambil sampel berulang kali dari data yang ada untuk mengestimasi distribusi probabilitasnya.

```
import random

def monte_carlo_bootstrap(trials=50000):
    data_asli =
    count_lebih_28 = 0
    total_samples = 0

    for _ in range(trials):
        # Mengambil satu data secara acak dari data asli (resampling with
        replacement)
        # Dalam konteks probabilitas empiris sederhana, ini konvergen ke
        frekuensi relatif
        sampel = random.choice(data_asli)
        total_samples += 1

        if sampel > 28:
            count_lebih_28 += 1

    prob = count_lebih_28 / total_samples
    print(f"Simulasi (Resampling) P(X > 28): {prob:.4f}")

    # Hitungan Eksak Python sebelumnya
    real_count = len([x for x in data_asli if x > 28])
    print(f"Hitungan Manual: {real_count/len(data_asli):.4f}")

monte_carlo_bootstrap()
```